

Passo a Passo para a Implementação do Bloco 4

1. **Criação da Classe de Reposição de Estoque (ReposicaoEstoque):**
 - Definir a lógica que **verifica o saldo total dos produtos no estoque** e sugere **ou gera ordens de compra automáticas** quando a quantidade cai abaixo de um limite crítico definido.
 -
2. **Geração de Dados Sintéticos de Teste (Módulo faker / Simulação):**
 - Implementar **rotinas para popular o sistema com comandas, produtos, lotes e pagamentos fictícios** de forma automatizada para validação dos fluxos.
 -
3. **Módulo de Relatórios e Análises (Relatorios):**
 - Criar **métodos para consolidar faturamento total, produtos mais vendidos, margem de lucro** (Preço de Venda vs. Preço de Compra dos Lotes) **e status atual do inventário**.
 -
4. **Persistência de Dados (Pickle / Armazenamento):**
 - **Implementar as funções salvar_estado() e carregar_estado() utilizando a biblioteca nativa pickle para salvar todo o estado do sistema (estoque, comandas e caixa) em arquivos .pkl antes do encerramento da aplicação.**

Relação do Bloco 4 com os Blocos Anteriores (1, 2 e 3)

- **Com o Bloco 1 (Produtos e Estoque):** A classe de reposição **analisa diretamente o saldo de cada Produto nos lotes de Estoque. Ao gerar novas compras, novos objetos LoteEstoque são criados** e inseridos ordenadamente na lista de lotes (garantindo o FIFO/PEPS). Além disso, os relatórios calculam os custos com base na `data_compra` e `preco_compra`.

- **Com o Bloco 2 (Comandas e Cozinha):** A geração automática de dados simula o consumo nas classes Comanda e ItemPedido. Os relatórios de consumo analisam os itens pedidos nas comandas para determinar os pratos e bebidas com maior giro de vendas.
- **Com o Bloco 3 (Caixa e Histórico):** O módulo de relatórios consolida o faturamento a partir das instâncias de Pagamento armazenadas na lista historico do Caixa. A persistência salva a lista completa de transações para que o caixa mantenha o histórico financeiro intacto entre execuções do programa.